

Name: Aziz Sayyad

Roll No: 381069

Batch: A3

Assignment No: 10

Problem Statement:

Write a better auto-complete algorithm using an N-gram model (similar models are used for translation, determining the author of a text, and speech recognition).

Objective:

- To understand the concept of N-gram language models and their role in predicting word sequences.
- To build a bigram (2-gram) frequency model from a text corpus using NLTK and Python's Counter.
- To compute conditional bigram probabilities to represent the likelihood of a word following a given preceding word.
- To implement an auto-complete function that predicts the most probable next word given an input string.
- To test and evaluate the auto-complete system on various input words and analyze the predictions.

S/W Packages and H/W apparatus used:

- Operating System: Windows/Linux/MacOS
- Kernel: Python 3.x
- Tools: Google Colab / Jupyter Notebook
- Hardware: CPU with minimum 4GB RAM
- Libraries and packages used: NLTK (punkt, punkt_tab, ngrams), Python collections (defaultdict, Counter)

Theory:

Language modeling is the task of assigning a probability to a sequence of words or predicting the next word given a preceding context. N-gram language models are among the oldest and most foundational approaches, underpinning applications such as auto-complete, speech recognition, machine translation, and authorship attribution.

N-gram Models:

An N-gram is a contiguous sequence of N tokens from a text. For $N=1$, we get unigrams (individual words); for $N=2$, bigrams (word pairs); for $N=3$, trigrams (word triples), and so on. An N-gram language model estimates the probability of a word given the preceding $N-1$ words using the Markov assumption - that the probability of the next word depends only on the immediately preceding $N-1$ words, not the entire history. This simplification makes computation tractable while still capturing meaningful local context.

Bigram Model:

A bigram model estimates $P(w_2 | w_1)$ -the probability of word w_2 appearing immediately after word w_1 . This is computed using Maximum Likelihood Estimation (MLE): $P(w_2 | w_1) = \text{Count}(w_1, w_2) / \text{Count}(w_1)$. For example, if 'artificial' is followed by 'intelligence' 3 times and 'neural' once in the corpus, then $P(\text{intelligence} | \text{artificial}) = 0.75$ and $P(\text{neural} | \text{artificial}) = 0.25$. The conditional frequency table is stored as a nested dictionary where `model[w1][w2]` holds the transition count.

Auto-complete with N-grams:

For auto-complete, given an input phrase, the last word is extracted and used to look up the bigram probability table. The candidate next word with the highest conditional probability - $\text{argmax}_{\{w_2\}} P(w_2 | w_1)$ -is returned as the prediction. This greedy selection returns the single most likely next word; more sophisticated approaches use beam search to maintain multiple candidate completions simultaneously.

Limitations of Basic N-gram Models and Improvements:

Basic N-gram models suffer from the zero-probability problem: if an N-gram was not observed in the training corpus, its probability is zero, causing the entire sentence probability to collapse to zero. Smoothing techniques address this -Laplace (add-1) smoothing adds 1 to every bigram count, Kneser-Ney smoothing redistributes probability mass to unseen N-grams based on context diversity, and interpolation combines unigram, bigram, and trigram probabilities for more robust estimation.

Methodology:

- Import NLTK, `nltk.util.ngrams`, `word_tokenize`, and Python's `defaultdict` and `Counter`.
- Download NLTK `punkt` and `punkt_tab` tokenizers.
- Define a 5-sentence NLP-themed corpus covering language models, auto-complete, N-grams, translation, and AI.
- Tokenize the entire corpus to lowercase using `word_tokenize()` to produce a flat token list.
- Generate all bigrams from the token list using `nltk.util.ngrams(tokens, 2)` and count their frequencies using `Counter`.
- Build a conditional frequency table (`defaultdict` of `Counters`) mapping each word w_1 to the counts of all words w_2 that follow it in the corpus.
- Compute conditional bigram probabilities by dividing each $w_1 \rightarrow w_2$ count by the total count of all words following w_1 , storing results in a probability dictionary.
- Implement `predict_next_word(input_text)`: tokenize the input, extract the last word, look up the bigram probability table, and return the word with the maximum conditional probability.
- Test the function on four input words -'artificial', 'language', 'auto', 'speech' -and print the predicted next word for each.

Use Cases:

- Use Case 1 - Mobile Keyboard Auto-complete: N-gram models power next-word prediction on smartphone keyboards. After the user types 'natural', the model predicts 'language' as the most likely continuation, reducing typing effort and improving speed.
- Use Case 2 - Authorship Attribution: By computing the probability of a text under N-gram models trained separately on different authors' writing styles, the model with the highest likelihood indicates the most probable author, useful in forensic linguistics and plagiarism detection.
- Use Case 3 - Speech Recognition Post-processing: After acoustic models decode phoneme sequences into candidate words, N-gram language models re-rank the candidates based on

linguistic plausibility, selecting word sequences with the highest N-gram probability to improve transcription accuracy.

Advantages:

- N-gram models are simple, interpretable, and computationally efficient to build and query.
- No neural network training is required; the model is built directly from frequency counts in the corpus.
- The conditional probability table provides a transparent and explainable representation of word transition likelihoods.
- The model is easily extensible to trigrams or higher-order N-grams for capturing longer context.
- Works entirely offline and on minimal hardware, making it suitable for embedded and resource-constrained applications.

Limitations:

- Bigram models only consider one preceding word; they cannot capture long-range dependencies in language.
- Zero-probability problem: words or bigrams absent from the training corpus cannot be predicted without smoothing.
- Performance degrades severely on small corpora where most bigrams are observed only once.
- Vocabulary size grows quadratically with N, causing memory and sparsity issues for higher-order N-grams.
- Modern neural language models (GPT, BERT) significantly outperform N-gram models by learning contextual representations from large corpora.

Applications:

- Search engine query auto-completion: N-gram models suggest popular query continuations as users type in the search box.
- Email and document writing assistants: Predicting the next word or phrase reduces typing effort in productivity tools.
- Machine translation: N-gram language models score translation candidates by linguistic fluency on the target side.
- Speech-to-text systems: N-gram language models improve word error rates by re-ranking acoustically similar word sequences.
- Code auto-complete: N-gram models trained on source code corpora predict the next token in programming environments (early versions of IntelliSense and similar tools).

Conclusion:

This assignment implemented a bigram-based N-gram language model for auto-complete using NLTK and Python. A five-sentence NLP corpus was tokenized, bigram frequencies were counted, and conditional probabilities $P(w_2 | w_1)$ were computed using Maximum Likelihood Estimation. The `predict_next_word()` function successfully predicted contextually accurate next words for all four test inputs: 'artificial' -> 'intelligence', 'language' -> 'processing', 'auto' -> 'complete', and 'speech' -> 'recognition'. The experiment illustrates the core principle of statistical language modeling -predicting the next word based on learned corpus statistics -and establishes the foundation for understanding more advanced language models used in translation, speech recognition, and modern AI text generation systems.